

Iterative (Looping) Workflows in LangGraph

Building self-correcting graphs that generate, evaluate, and refine their own output — through the lens of a real Generator → Evaluator → Optimizer loop that writes viral X (Twitter) posts.

WHAT THIS CHAPTER COVERS

- ▶ What iterative / looping workflows are
- ▶ The Generate → Evaluate → Optimize pattern
- ▶ Designing state for a multi-LLM loop
- ▶ Reducers for history tracking (operator.add)
- ▶ Structured output for deterministic routing
- ▶ Conditional edges that create real loops
- ▶ Guarding against infinite loops (max_iterations)
- ▶ Building and running the full graph

Iterative (Looping) Workflows in LangGraph

RECAP — CHAPTERS 5–7

So far this series has covered three workflow types: **Sequential** (tasks executed one after another in a straight line), **Parallel** (independent tasks fanned out and merged back into shared state via reducers), and **Conditional** (a single branch chosen out of several, based on a routing function). This chapter introduces the fourth and final foundational pattern: **Iterative (Looping) Workflows** — graphs that deliberately revisit the same nodes to progressively improve a result.

1. What Is an Iterative (Looping) Workflow?

Definition

An *iterative workflow* (also called a *looping workflow*) is a LangGraph pattern in which execution deliberately cycles back through one or more already-visited nodes in order to improve a result, rather than always moving strictly forward. Unlike a sequential chain, the path through the graph is not fixed in length — the same pair of nodes may run once, three times, or five times, depending on how the intermediate output is judged.

Why It Exists

A single call to an LLM rarely produces a maximally polished result on the first attempt. Ask a model to write a tweet, a summary, or a piece of code, and the first draft is usually mediocre — serviceable, but not the best the model is capable of. Iterative workflows exist to close that gap: instead of accepting the first draft, the graph judges it against explicit criteria and, if it falls short, sends it back for improvement — automatically, without a human re-prompting the model by hand.

Problem It Solves

Iterative workflows solve the **quality-on-first-attempt problem**. Any pipeline that blindly accepts an LLM's first output risks shipping repetitive, generic, or low-quality content. By wrapping generation in a judge-and-refine loop, the workflow keeps producing new attempts until a defined bar is cleared — or a safety limit is hit.

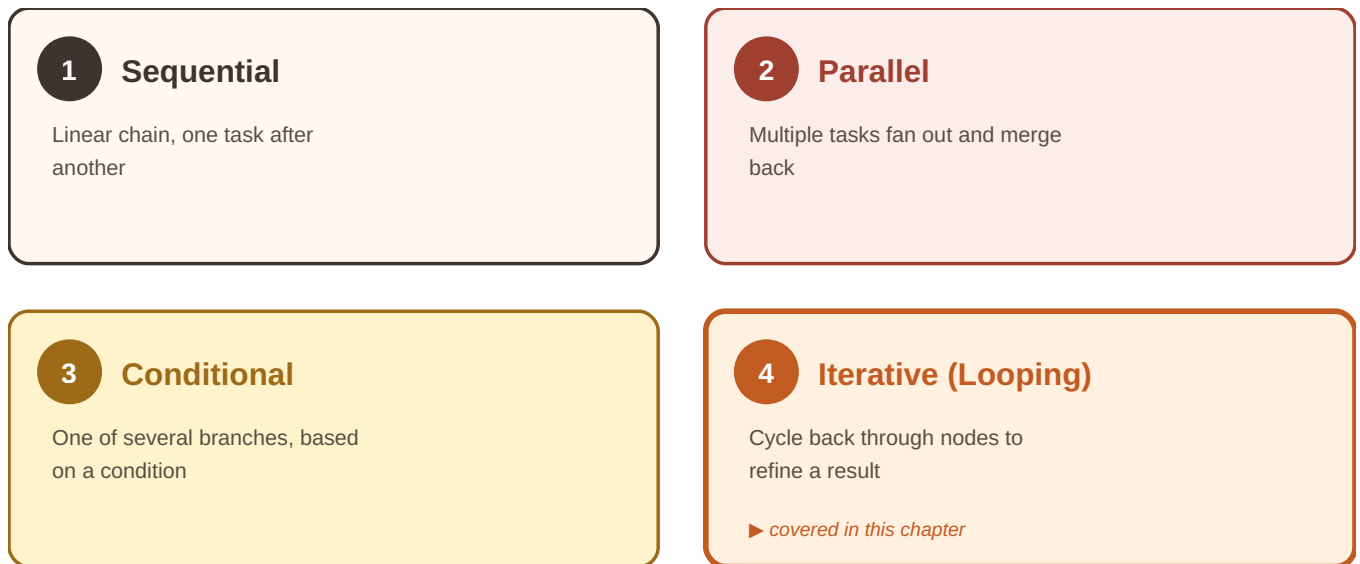


Figure 8.1 — The four workflow patterns covered in this series; iterative workflows are the focus of this chapter.

How It Works — The Generate → Evaluate → Optimize Pattern

The canonical shape of an iterative workflow has three roles:

1. GENERATE

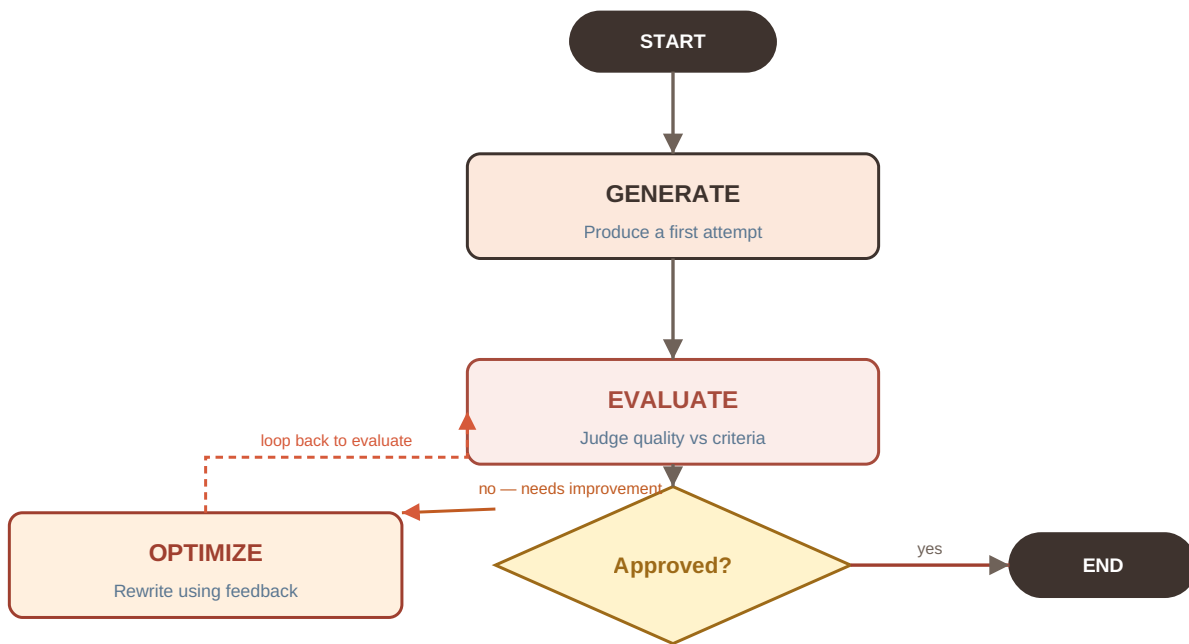
Produce a first attempt at the task.

2. EVALUATE

Judge that attempt against explicit, written criteria, and decide: is this good enough to ship, or does it need work?

3. OPTIMIZE

If it needs work, rewrite the attempt using the evaluator's feedback, then send the new attempt **back to Evaluate** — not back to Generate. The loop is between Evaluate and Optimize; Generate only ever runs once, right at the start.



The Evaluate → Optimize → Evaluate cycle repeats until the result is approved or a maximum iteration limit is reached.

Figure 8.2 — Abstract shape of an iterative workflow: a single Generate step feeds an Evaluate ↔ Optimize loop that exits only on approval.

IMPORTANT

The loop is **not** Generate → Evaluate → Generate → Evaluate. Re-generating from scratch each time throws away everything the evaluator already learned about the attempt. A correct iterative workflow loops between **Evaluate** and **Optimize**: Optimize always has both the previous attempt and the specific feedback on it, so each new version builds on the last instead of starting over.

🔄 Quick Revision — Iterative Workflows

- An iterative/looping workflow deliberately revisits nodes to refine a result rather than always moving forward.
- It exists because a single LLM pass rarely produces the best possible output on the first try.
- The canonical shape is Generate (once) → Evaluate ↔ Optimize (repeats).
- The loop runs between Evaluate and Optimize, never back through Generate.
- This is the fourth workflow pattern in the series, after Sequential, Parallel, and Conditional.

2. Real-World Use Case: An Automated X (Twitter) Post Generator

Motivation

The chapter builds its iterative workflow around a genuine problem: producing consistent, good content on a platform the author does not have time to be active on. YouTube already consumes most available content-creation time, leaving little bandwidth for platforms like X (Twitter). An automated workflow that takes a topic and produces a ready-to-post tweet would recover that time — but only if the output is actually good. A fully automated pipeline with no quality gate risks publishing repetitive, mediocre posts that create no value for readers.

Problem It Solves

The core difficulty is **evaluating LLM output automatically**. Asking an LLM once for a tweet on a topic rarely produces a genuinely funny, original, first-attempt-perfect result. An iterative workflow addresses this directly: a second LLM acts as a strict critic, and a third LLM rewrites the post using that critic’s feedback, repeating the cycle until the post is approved.

How It Works — End-to-End Flow

1. A **topic** is supplied to the workflow (e.g. “Indian Railways”).
2. A **generator LLM** writes a first tweet on that topic.
3. An **evaluator LLM**, given a strict, explicit rubric, judges the tweet and returns either **approved** or **needs_improvement**, plus written feedback.
4. If **approved**, the workflow ends (in a production system, a human-in-the-loop step and a platform API call would follow).
5. If it **needs improvement**, an **optimizer LLM** rewrites the tweet using the topic, the previous tweet, and the evaluator’s feedback, and sends the new version back to the evaluator.
6. This Evaluate → Optimize cycle repeats until the tweet is approved or a maximum iteration count is reached.

REAL WORLD INSIGHT

Splitting generation, evaluation, and optimization into three separate LLM calls — rather than one LLM trying to self-critique in a single pass — mirrors how human editorial workflows work: a writer, an editor, and a rewrite pass each focus on one job. Giving the evaluator a very explicit, narrow rubric (what counts as funny, what format to auto-reject) matters more for output quality than which specific model is used.

Quick Revision — The Use Case

- The workflow generates X (Twitter) posts on a given topic, targeting short, funny, original tweets.
- Its purpose is to save the creator’s time while still producing genuinely good, non-repetitive content.
- Three components: Generation, Evaluation, Optimization.
- The evaluator is deliberately given a strict, explicit rubric — vague criteria produce vague quality control.
- In a real deployment, an approved post would still pass through a human-in-the-loop check before publishing.

3. Workflow Architecture: Three Specialized LLMs

Definition

The workflow uses **three separate LLM instances**, one per role, rather than reusing a single model for every step. In an ideal build, each LLM would be chosen for its strength at that specific job; the notebook uses Groq-hosted open models for all three, with a deliberately weaker model for generation so that the evaluation/optimization loop has something meaningful to do.

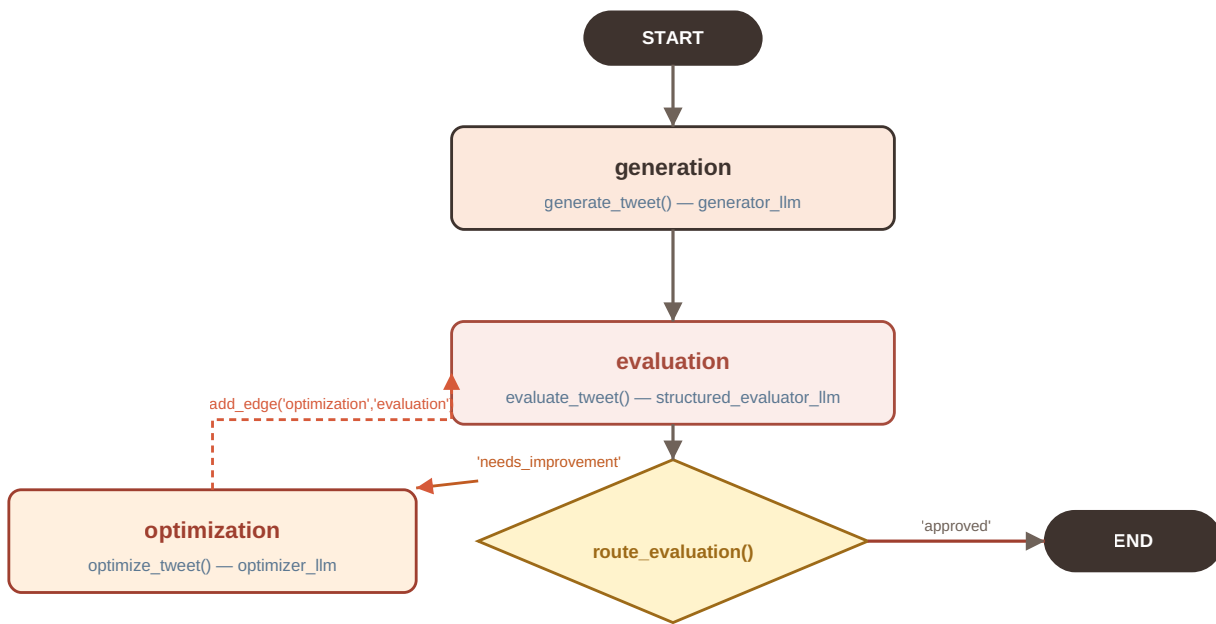
```
# three distinct LLM roles
generator_llm = ChatGroq(
    model = "openai/gpt-oss-20b"
)

evaluator_llm = ChatGroq(
    model = "openai/gpt-oss-120b"
)

optimizer_llm = ChatGroq(
    model = "openai/gpt-oss-120b"
)
```

Role of Each LLM

LLM	Job	Ideal Trait
Generator	Produce the first draft of the tweet from the topic	Strong, creative writing ability
Evaluator	Judge the tweet against explicit criteria; return a structured verdict + feedback	Follows instructions strictly ("religiously")
Optimizer	Rewrite the tweet using the previous attempt and the evaluator's feedback	Strong revision / rewriting ability



Conditional edge (evaluation → route_evaluation): "approved" → END | "needs_improvement" → optimization.
 Loop edge (optimization → evaluation) is what keeps the cycle running until approval or max_iterations.

Figure 8.3 — The concrete X post generator graph: generation → evaluation → conditional routing → END or optimization → back to evaluation.

BEST PRACTICE

Use a deliberately narrower or weaker model for generation only while prototyping, so the evaluator has genuine work to reject and you can observe the loop actually cycling. In production, pick a strong writing-oriented model for generation and a strict, instruction-following model for evaluation — matching model strengths to each node's job, rather than reusing one model everywhere.

Quick Revision — Architecture

- Three distinct LLM instances: generator_llm, evaluator_llm, optimizer_llm.
- generator_llm should favor writing/creative strength; evaluator_llm should favor strict instruction-following.
- optimizer_llm rewrites using both the original tweet and the specific feedback — not from scratch.
- Node names in the graph: 'generation', 'evaluation', 'optimization'.

4. State Design: TweetState and Reducers

Definition

Every LangGraph workflow needs a state schema defined before any nodes are built. For this workflow, the state must capture the topic, the current tweet, the evaluator's verdict and feedback, an iteration counter and its cap, and — because

the chapter also wants to inspect every intermediate draft — a running history of every tweet and every feedback string produced.

```
class TweetState(TypedDict):
    topic : str
    tweet : str
    evaluation : Literal['approved', 'needs_improvement']
    feedback : str
    iterations : int
    max_iterations : int

    tweet_history : Annotated[list[str], operator.add]
    feedback_history : Annotated[list[str], operator.add]
```

Field-by-Field Breakdown

PLAIN FIELDS (OVERWRITTEN ON EVERY UPDATE)

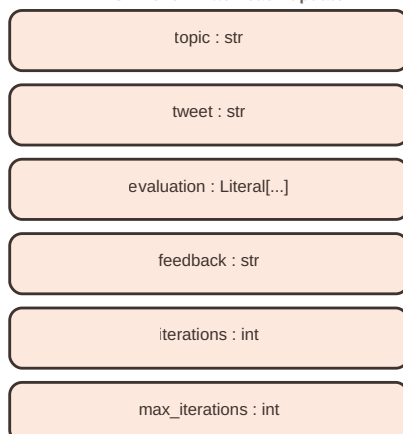
- **topic** — the user-supplied subject the tweet must be about.
- **tweet** — the current draft of the tweet (latest generated or optimized version).
- **evaluation** — the evaluator's latest verdict, a `Literal` restricted to two values.
- **feedback** — the evaluator's latest written feedback string.
- **iterations** — how many Evaluate/Optimize cycles have run so far.
- **max_iterations** — the hard cap used to break out of the loop (set to 5 in this chapter).

REDUCER FIELDS (MERGED ACROSS UPDATES)

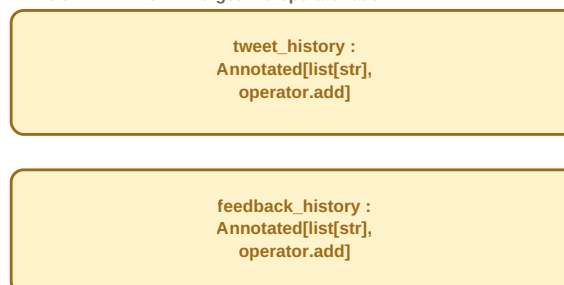
tweet_history and **feedback_history** use `Annotated[list[str], operator.add]`. This is the same reducer pattern from the Parallel Workflows chapter: every time a node returns a value for one of these keys, LangGraph **appends** it to the existing list instead of replacing it, because each node returns a single-item list (e.g. `{"tweet_history": [response]}`) and `operator.add` concatenates lists.

TweetState (TypedDict)

PLAIN FIELDS — overwritten each update



REDUCER FIELDS — merged via operator.add



Each node returns a single-item list; the reducer appends it instead of replacing the list, building a running history across iterations.

Figure 8.4 — *TweetState's plain fields (overwritten each update) vs. its reducer fields (appended across the whole loop).*

COMMON MISTAKE

The notebook's `TweetState` defines `evaluation` as `Literal['approved', 'needs_imporvement']` — note the typo, *imporvement* instead of *improvement* — while the Pydantic evaluation schema and the routing function both correctly use `'needs_improvement'`. This inconsistency does not break anything at runtime because Python's `TypedDict/Literal` annotations are not enforced automatically; but it is exactly the kind of typo that silently defeats type-checking tools if you rely on them. Keep string literals used for control flow consistent everywhere they appear.

🔄 Quick Revision — State & Reducers

- `TweetState` is a `TypedDict` with six plain fields plus two reducer-backed history lists.
- Plain fields (`topic`, `tweet`, `evaluation`, `feedback`, `iterations`, `max_iterations`) are overwritten each update.
- `tweet_history` and `feedback_history` use `Annotated[list[str], operator.add]` to accumulate across iterations.
- Each node must return its history update as a single-item list, e.g. `{'tweet_history': [response]}`.
- `max_iterations` exists specifically to guarantee the loop terminates.

5. Building the Nodes

Node 1 — `generate_tweet`

The generator node builds a prompt using `SystemMessage` and `HumanMessage` (the message-list pattern from the LangChain playlist), instructs the model to write a short, original, funny tweet on the topic with explicit formatting rules, invokes `generator_llm`, and returns the result into both `tweet` and the first entry of `tweet_history`.

```
def generate_tweet(state : TweetState):

    # Prompt
    messages = [
        SystemMessage(content="You are a funny and clever Twitter/X influencer."),
        HumanMessage(content=f"""
            Write a short, original, and hilarious tweet on the topic: "{state['topic']}".
            Rules:
            - Do NOT use question-answer format.
            - Max 280 characters.
            - Use observational humor, irony, sarcasm, or cultural references.
            - Think in meme logic, punchlines, or relatable takes.
            - Use simple, day to day english
            """)
    ]

    # Send to generator
    response = generator_llm.invoke(messages).content

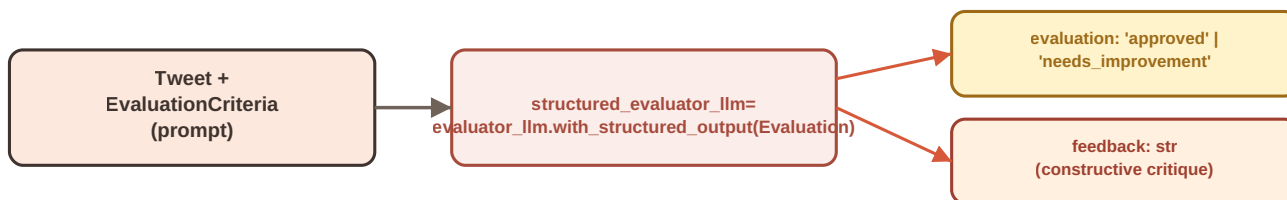
    return {'tweet':response, 'tweet_history':[response]}
```

Node 2 — evaluate_tweet

The evaluator node needs a response shape the graph can act on programmatically — not free-form prose. This is done with a **Pydantic schema** and `with_structured_output`, forcing every evaluator response into exactly two fields: evaluation and feedback.

```
class Evaluation(BaseModel):
    evaluation : Literal['approved', 'needs_improvement'] = Field(...,description = "This
    represents whether the tweet is approved for posting or not.")
    feedback : str = Field(...,description = "Constructive feedback of the tweet post.")

structured_evaluator_llm = evaluator_llm.with_structured_output(Evaluation)
```



Why structured output?

Without a Pydantic schema, the LLM's reply is free-form text that `route_evaluation()` cannot reliably parse. `with_structured_output(Evaluation)` forces every response into the exact evaluation / feedback shape the graph's conditional-edge function depends on — this is what makes the routing decision deterministic.

Figure 8.5 — `with_structured_output(Evaluation)` forces the evaluator's reply into a fixed, parseable shape that the conditional-edge function can act on.

INTERVIEW TIP

If asked why the evaluator node uses structured output but the generator node does not, the answer is about **what downstream code depends on**: the tweet itself just needs to be text a human or an API can post, but the evaluator's output feeds directly into a conditional-edge function that branches on an exact string. Any prompt-based free-text reply is too unreliable for that; a validated schema is what makes the branch deterministic.

```

def evaluate_tweet(state : TweetState):

    # Prompt
    messages = [
        SystemMessage(content="You are a ruthless, no-laugh-given Twitter critic. You
evaluate tweets based on humor, originality, virality, and tweet format."),
        HumanMessage(content=f"""
            Evaluate the following tweet:

            Tweet: "{state['tweet']}"

            Use the criteria below to evaluate the tweet:

            1. Originality - Is this fresh, or have you seen it a hundred times before?
            2. Humor - Did it genuinely make you smile, laugh, or chuckle?
            3. Punchiness - Is it short, sharp, and scroll-stopping?
            4. Virality Potential - Would people retweet or share it?
            5. Format - Is it a well-formed tweet (not a setup-punchline joke, not a Q&A
joke, and under 280 characters)?

            Auto-reject if:
            It's written in question-answer format (e.g., "Why did ... " or "What happens
when ... ")
            It exceeds 280 characters
            It reads like a traditional setup-punchline joke
            Dont end with generic, throwaway, or deflating lines that weaken the humor

            ### Respond ONLY in structured format:
            - evaluation: "approved" or "needs improvement"
            - feedback: One paragraph explaining the strengths and weaknesses
            """)
    ]

    # Send to evaluator
    response = structured_evaluator_llm.invoke(messages)

    return {"evaluation":response.evaluation,
"feedback":response.feedback,"feedback_history":[response.feedback]}

```

Node 3 — optimize_tweet

The optimizer node receives the current tweet, the topic, and the evaluator's feedback, asks the optimizer LLM to rewrite the tweet accordingly, and — critically — increments the iteration counter each time it runs, since this is the only node where a new attempt is actually produced after the first.

```
def optimize_tweet(state : TweetState):

    # Prompt
    messages = [
        SystemMessage(content="You punch up tweets for virality and humor based on given feedback."),
        HumanMessage(content=f"""
            Improve the tweet based on this feedback:
            "{state['feedback']}"

            Topic: "{state['topic']}"
            Original Tweet:
            {state['tweet']},

            Re-write it as a short, viral-worthy tweet. Avoid Q&A style and stay under 280
            characters."""
        )
    ]

    # Send to Optimizer
    response = optimizer_llm.invoke(messages).content
    iteration = state['iterations'] + 1

    return {"tweet":response,"iterations":iteration,"tweet_history":[response]}
```

🔄 Quick Revision — The Three Nodes

- `generate_tweet`: builds a system+human prompt, invokes `generator_llm`, returns `tweet + tweet_history`.
- `evaluate_tweet`: uses `with_structured_output(Evaluation)` so the graph gets a guaranteed evaluation/feedback shape.
- `optimize_tweet`: rewrites using the previous tweet AND the specific feedback, and increments `'iterations'`.
- Both `generate_tweet` and `optimize_tweet` write into `tweet_history`; only `evaluate_tweet` writes into `feedback_history`.
- Only `optimize_tweet` increments the iteration counter — generation counts as the implicit first attempt.

6. Conditional Routing and the Loop Guard

Definition

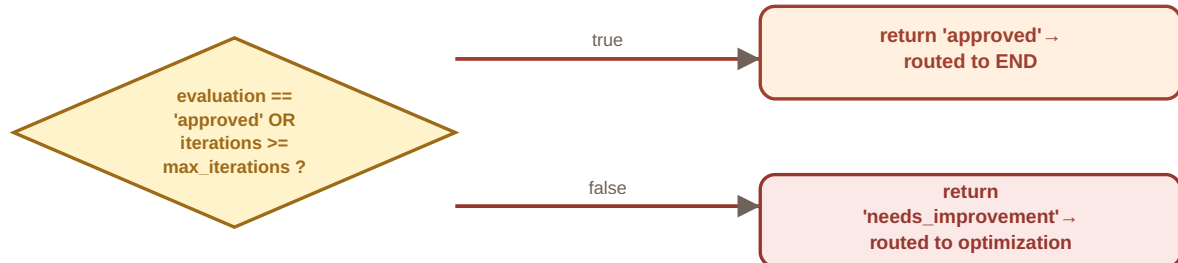
The loop itself is created purely through **how edges are wired** — no special "loop" API exists in LangGraph. A conditional edge out of `evaluation` decides, on every pass, whether to route to `END` or back to `optimization`; a normal edge from `optimization` back to `evaluation` is what actually closes the cycle.

The Routing Function

`route_evaluation` returns `'approved'` under **two** conditions, not one: either the evaluator genuinely approved the tweet, **or** the iteration cap has been reached. This second condition is the loop's safety valve.

```
def route_evaluation(state : TweetState):
    if state['evaluation'] == 'approved' or state['iterations'] >= state['max_iterations']:
        return 'approved'
    else:
        return 'needs_improvement'
```

route_evaluation(state)



Iteration counter (safety guard)

optimize_tweet() increments iterations by 1 on every pass: `iteration = state['iterations'] + 1`.
Without this cap, a stubborn evaluator + a weak generator could loop indefinitely — `max_iterations` forces an exit.

Figure 8.6 — `route_evaluation()`'s two-condition guard: approval OR exhausted iterations both route to END.

COMMON MISTAKE

It is easy to forget the `iterations >= max_iterations` half of this condition and route purely on the evaluator's verdict. If the evaluation criteria are too strict for the model actually generating and optimizing the tweet, the workflow can cycle indefinitely, repeatedly rejecting every new draft. The iteration cap is not optional defensive code — without it, a harsh evaluator paired with a weak generator/optimizer pair can hang the workflow.

Wiring the Edges

With all three node functions and the routing function defined, the graph is assembled with four edge statements: one plain start edge, one plain edge from generation to evaluation, one conditional edge out of evaluation, and one plain edge closing the loop from optimization back to evaluation.

```
graph = StateGraph(TweetState)

graph.add_node("generation", generate_tweet)
graph.add_node("evaluation", evaluate_tweet)
graph.add_node("optimization", optimize_tweet)

graph.add_edge(START, 'generation')
graph.add_edge('generation', 'evaluation')

graph.add_conditional_edges('evaluation', route_evaluation,
{'approved': END, 'needs_improvement': 'optimization'})

graph.add_edge('optimization', 'evaluation')
```

IMPORTANT

Creating a loop in LangGraph is nothing more than **edge manipulation**: `add_conditional_edges` from evaluation gives the exit condition, and a plain `add_edge('optimization', 'evaluation')` gives the return path. There is no separate looping construct — any cycle in the graph's edge structure behaves as a loop for as long as the routing logic keeps choosing to re-enter it.

🔄 Quick Revision — Routing & the Loop

- Loops in LangGraph are created purely by edge wiring — there is no dedicated 'loop' construct.
- `route_evaluation()` returns 'approved' if the evaluator approved OR the iteration cap is reached.
- `add_conditional_edges('evaluation', route_evaluation, {...})` creates the exit/continue branch.
- `add_edge('optimization', 'evaluation')` is the single edge that turns the branch into an actual loop.
- Always pair a conditional loop-continue branch with an iteration/attempt cap to guarantee termination.

7. Assembling and Running the Workflow

Compiling and Invoking

Once nodes and edges are in place, the graph is compiled into a runnable workflow and invoked with an initial state supplying the topic, a starting iteration count of 1, and the iteration cap.

```
workflow = graph.compile()
workflow
```

```
initial_state = {
    "topic": "Indian Railways",
    "iterations": 1,
    "max_iterations": 5
}
result = workflow.invoke(initial_state)

result
```

Observed Behaviour

Running the workflow on the topic “*Indian Railways*” produced a tweet comparing boarding a train to a trip to Hogwarts — approved on the first pass with the stronger generator model. Switching the generator to a smaller/weaker model made the evaluator reject the first draft and approve a rewritten second draft instead, demonstrating the loop actually cycling through Evaluate → Optimize → Evaluate before exiting.

BEST PRACTICE

While developing an iterative workflow, deliberately weaken the generator (or feed an unusual/difficult topic) so you can observe the loop running more than one iteration before you trust it in production. If every test run is approved on the first pass, you have not actually exercised the Evaluate/Optimize cycle or verified the loop-back edge works correctly.

Inspecting Iteration History

Because `tweet_history` and `feedback_history` accumulate across every pass through the loop, the full sequence of draft tweets and the feedback that produced each rewrite can be inspected after the run completes — useful for debugging why the evaluator rejected earlier attempts.

```
# print every tweet produced across all iterations
for tweet in result['tweet_history']:
    print(tweet)
```

🔄 Quick Revision — Assembly & Execution

- `graph.compile()` turns the `StateGraph` into a runnable workflow object.
- `invoke()` needs at minimum topic, iterations (start at 1), and `max_iterations` in the initial state.
- A stronger generator model can get approved on the very first pass — no loop iterations needed.
- A weaker generator model demonstrates the loop: reject → optimize → re-evaluate → approve.
- `tweet_history` / `feedback_history` hold every draft and its feedback, useful for debugging the loop.

K. Key Takeaways

- **Iterative workflows** deliberately cycle through nodes to refine a result, using the Generate (once) → Evaluate ↔ Optimize (repeats) pattern.
- **Three specialized LLMs** — generator, evaluator, optimizer — each handle one job, matching model strengths to node responsibilities.
- **State design** combines plain overwritten fields (topic, tweet, evaluation, feedback, iterations, max_iterations) with reducer-backed history lists (tweet_history, feedback_history via operator.add).
- **Structured output** (with_structured_output + a Pydantic schema) turns the evaluator's free-form judgment into a guaranteed, parseable evaluation/feedback shape the graph can branch on.
- **Loops are pure edge wiring**: a conditional edge out of evaluation decides END vs. optimization, and a plain edge from optimization back to evaluation closes the cycle.
- **The iteration guard** (iterations >= max_iterations inside the routing function) is what guarantees the loop terminates, independent of whether the evaluator ever approves.
- **optimize_tweet** is the only node that increments the iteration counter, since it is the only node that runs on every repeat of the loop.

R. Revision Sheet

Iterative Workflow

A LangGraph pattern where execution deliberately revisits nodes to improve a result, rather than always moving forward.

Generate → Evaluate → Optimize

Generate runs once; Evaluate and Optimize repeat in a loop until approval or a max-iteration cutoff.

Reducer History Fields

Annotated[list[str], operator.add] fields (tweet_history, feedback_history) accumulate one entry per node call instead of being overwritten.

with_structured_output

Binds a Pydantic schema to an LLM so every response is forced into a fixed, parseable shape — essential for routing logic.

route_evaluation()

Conditional-edge function returning 'approved' or 'needs_improvement'; approves either on genuine approval or on hitting max_iterations.

add_conditional_edges

graph.add_conditional_edges(node, routing_fn, {value: target_node, ...}) — the branch mechanism that makes the loop possible.

Loop-Closing Edge

A single plain add_edge('optimization','evaluation') is what turns a branch into an actual repeating cycle.

Iteration Guard

max_iterations plus an iterations counter incremented in the optimize node prevents infinite looping.

I. Interview Questions

Q1. What is an iterative (looping) workflow in LangGraph, and how does it differ from a sequential workflow?

Q2. Why does the loop in a Generate → Evaluate → Optimize pattern run between Evaluate and Optimize, rather than back through Generate?

Q3. How do you actually create a "loop" in LangGraph, given there is no dedicated loop API?

Q4. Why is `with_structured_output` used for the evaluator node but not for the generator or optimizer nodes?

Q5. What problem would arise if `route_evaluation()` only checked `state['evaluation']` and ignored the iteration count?

Q6. Explain the difference between a plain `TypedDict` field and a field annotated with `Annotated[list[str], operator.add]` in LangGraph state.

Q7. Why must a node return a single-item list (e.g. `{"tweet_history": [response]}`) rather than the raw string when writing to a reducer field?

Q8. In this workflow, which node increments the iteration counter, and why does only that node do so?

Q9. What is the practical purpose of using three separate LLM instances instead of one shared LLM for generation, evaluation, and optimization?

Q10. Describe what would happen at runtime if the evaluator's Pydantic schema used the literal `'needs_improvement'` but the routing function's dictionary key was misspelled as `'needs_imporvement'`.

F. Further Reading

- LangGraph official documentation — Conditional Edges and Branching
- LangGraph official documentation — State, Reducers, and Annotated Types
- LangChain documentation — Structured Output (`with_structured_output`)
- Pydantic documentation — `BaseModel` and `Field` for schema definition
- Anthropic / OpenAI prompt engineering guides — writing explicit evaluation rubrics for LLM-as-judge patterns